Documentação do circuito e arquitetura de software de movimentação da mesa

Felipe Nanini Cavelagna

3 de Julho de 2018

Este documento refere-se à implementação de um motor controlado pelo drive e indexador OEM 750X-M2 para a movimentação da mesa do túnel de vento do IPT, figura 1, com o objetivo de abranger a arquitetura de software implementada, scripts e circuito.

1 API

A definição geral de um eixo fica com a seguinte estrutura de informações:

- Nome do eixo;
- Limites superior e inferior do eixo;
- Sensor de homing para o eixo;

Figura 1: Sistema de transmissão para a movimentação da mesa.



- Sensores de fim de curso para o eixo;
- Passo por motor ou encoder;

Desses itens, apenas a informação dos sensores de fim de curso não apresentam uma alteração para a definição da interface. Entretanto, todos devem ser implementados no script que lida com a parte de mais baixo nível de acordo com o hardware a ser implementado.

Em relação às funções definidas para a movimentação e supervisão do sistema, os itens a seguir representam a apresentação gerala da API:

- rmove(self, x = ‘’, y = ‘’, z = ‘’, sync = False) – realiza o movimento relativo do seu sistema.
- move(self, x = ‘’, y = ‘’, z = ‘’, a = False, r = False, sync = False) – realiza o movimento absoluto.
- abs_position(self, pulses=False) – Retorna a posição absoluta de cada eixo.
- position(self) – Retorna a posição de cada eixo.
- home(self, eixo = ‘’, sinal = ‘’) – Se o eixo estiver equipado com sensores de homing, envia o sinal para o inicio dessa operação.
- set_reference(self) – Define o ponto atual como referência.
- set_abs_reference(self) – Define o ponto atual como referência absoluta.
- stop(self) – Interrompe todos e quaisquer movimentos do sistema.
- clear(self) – Limpa a memória.
- waitUntilDone(self, wait=0.01) – Realiza um movimento síncrono.

1.1 Controle por meio do OEM-750X-M2

Os parâmetros necessários para a inicialização da classe que comunica ao sistema de movimentação são:

port – Indica a porta serial em que o indexador está conectado;
baudrate – Indica a taxa de envio de dados;
bytesize – Indica a quantidade de bits de dados;
parity – Habilita o bit de paridade;
stopbits – número de bits de parada;

passoMotor – Define um número de calibração para os passos que o motor deve percorrer ,de acordo com a necessidade do movimento;

passoEncoder – (Opcional) Define um número de calibração para os passos que o encoder deve medir, de acordo com a necessidade do movimento.

O controle feito pelo driver/indexador e feito para a rotação e a elevação da mesa e, portanto, e definido na opção padrão presentes na interface e pela atribuicao das strings "rotacao"e "elevacao"ao parâmetro "padrao".

As funções serao descritas na sequencia.

- rmove(self, x, y, z, sync = False)

  x – deslocamento de acordo com a unidade de medida para a qual os passos foram calibrados

  sync – Habilita a movimentação síncrona

  Realiza o movimento relativo a posição atual, movendo x de acordo com a unidade de medida para a qual os passos foram calibrados.

- move(self, x, a = False, r = False, sync = False)

  x – deslocamento de acordo com a unidade de medida para a qual os passos foram calibrados

  a – Habilita a referência absoluta

  r – Habilita o movimento relativo

  sync – Habilita a movimentação síncrona

  Realiza o movimento para a posição x em relação a referência do sistema.

- abs_position(self, pulses=False)

  pulses – Retorna a posição de acordo com os passos percorridos pelo motor

  Retorna a posição relativa à referência absoluta.

- position(self)

  Retorna a posição relativa à referência.

- home(self, sinal, eixo)

  sinal – Define a direção a ser percorrida

  axis – Indica o eixo a ser percorrido

  Realiza o movimento no eixo e direção indicados até o sensor de homing.

- set_abs_reference(self)

  Define o ponto atual como referência absoluta.

- set_reference(self)

  Define o ponto atual como referência.

- stop(self)

  Interrompe qualquer movimento.

- clear(self)

  Reseta as configurações do indexador.

- step_motor(self)

  Habilita o movimento de acordo com o passo calibrado para o motor.

- step_encoder(self)

  Habilita o movimento de acordo com o passo calibrado para o encoder.

2 Arquitetura de Software

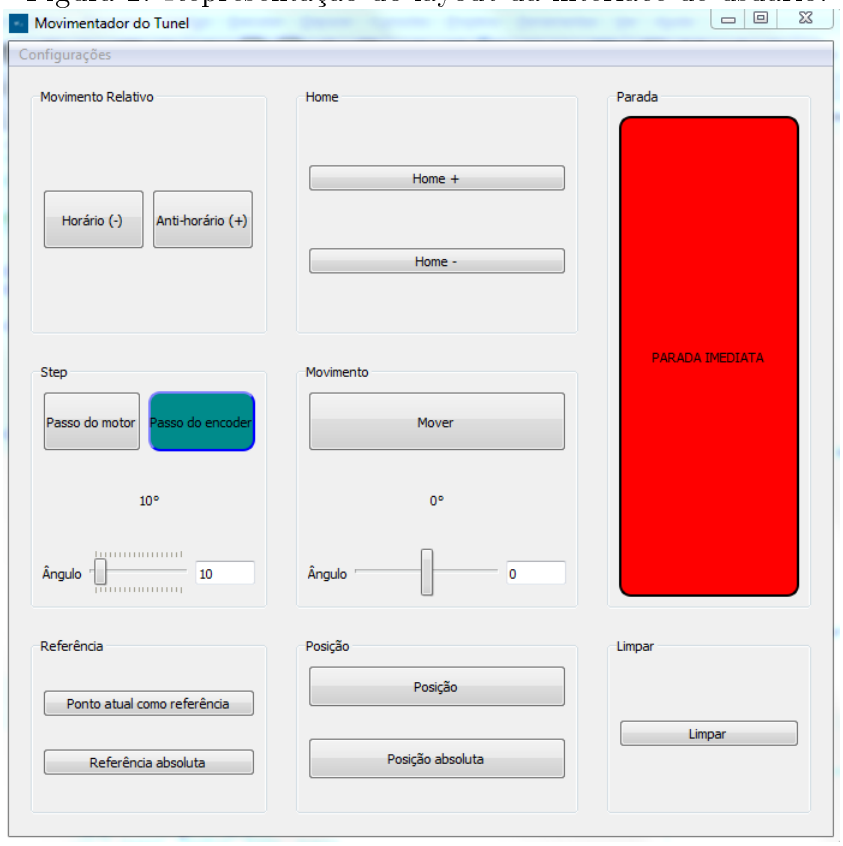
A arquitetura é composta pelo drive e indexador OEM 750X-M2 da Parker Motion, conectado ao computador industrial, o código Python para a interface e para a comunicação serial com o drive.

Especificando os elementos da arquitetura temos:

- O drive e indexador OEM 750X-M2 é o mesmo produto que os drives utilizados no sistema de movimentação do túnel (OEM 750), entretanto inclui o indexador, controlador de posição, o qual evita a necessidade de uma placa controladora como a ACR 1505 utilizada para o exemplo citado. Portanto, conta com as características do drive que garante os recursos básicos para a implementação de um sistema de controle de movimento. O indexador utiliza comandos da linguagem da Compumotor, *X Series Language* e provê entradas e saídas analógicas e digitais para controle e comunicação.

- O código Python utilizado é dividido em *pyqtmesa.py*, *mesa.py*, *mesaxmlrpc.py*. O arquivo *pyqtmesa.py* gera a interface de usuário por meio do PyQt5. Os scripts de *mesa.py* reúnem funções para a comunicação com o drive/indexador, utilizando o pacote pySerial, com as configurações da porta já feitas. Por fim, *mesaxmlrpc.py* faz a comunicação entre os dois últimos arquivos e fica à disposição para um possível acesso remoto.

Figura 2: Representação do layout da interface de usuário.



3 Código

O código referente à interface de usuário gera uma janela, apresentada na figura 2, cujos botões acionam as funções correspondentes no arquivo *mesa.py* enviando comandos de rotação, absoluta e incremental, nos sentidos horário e anti-horário, *homing*, parada imediata, definição de referência e verificação de posição.

Em relação aos comandos utilizados para a comunicação pelo protocolo RS-232, primeiramente é necessária a inicialização da porta serial de acordo com os parâmetros de comunicação listados abaixo

- Baud Rate: 9600
- Data Bits: 8
- Stop Bit: 1

- Parity: Nenhum

No código, o método .Serial do pacote pySerial permite as definições de cada um desses parâmetros, como mostra o código:

```
def __init__(self, passoMotor=28100000, passoEncoder=2492,
port = "\\\\.\\COM1", baudrate = 9600,
bytesize = serial.EIGHTBITS, parity = serial.PARITY_NONE,
stopbits = serial.STOPBITS_ONE):
"""Incializa a conexao com a porta serial

Parametros   Funcao
passoMotor   Define o valor de passos do motor para
             uma volta completa
passoEncoder Define o valor de passos do encoder para
             uma volta completa
port         A porta serial conectada
baudrate     Taxa em que os sinais da comunicacao variam
bytesize     Numero de bits por caracter (5, 6, 7, 8)
parity       Bit de verificacao de paridade (Nenhum-N,
             Par-E, Impar-O, Espaco-S, Marca-M)
stopbits     Bit ou bits de parada no final do
             caracter (1 ou 2)"""
self.ser = serial.Serial(port = port, baudrate = baudrate,
bytesize = bytesize, parity = parity, stopbits = stopbits,
timeout = 0.01)
time.sleep(1)
self.passoMotor = passoMotor/360
self.passoEncoder = passoEncoder/360
self.x0 = 0.0
self.stopPress = False
self.connect()
self.step = False
```

O valor definido na variável passoMotor indica o número de passos do motor para uma volta completa e o passoEncoder indica o número de passos do encoder para uma volta completa, os quais foram calibrados, e serão utilizados nas funções de movimentação e posição para a conversão da medida de rotação em graus. Os demais argumentos da função de inicialização da classe são responsáveis pela configuração da comunicação com a porta serial, cuja descrição está comentada no código.

Na sequência, devemos enviar a configuração inicial para a movimentação do motor, definindo assim a velocidade, aceleração, o modo de movimentação e os estados de configuração dos sensores. A função *connect* se encarrega de habilitar a comunicação e executa essas primeiras configurações.

```
def connect(self):
    """Estabelece a conexao a partir do terminal e
    inicializa o canal pelo prompt do programa"""
    self.isconnected = True
    self.sendData("E")
    reply = self.get_reply()
    seq = ["LD3", "MN", "OSC1" "A5", "V5", "ER1"]
    for i in seq:
        self.sendData(i)

    return reply
```

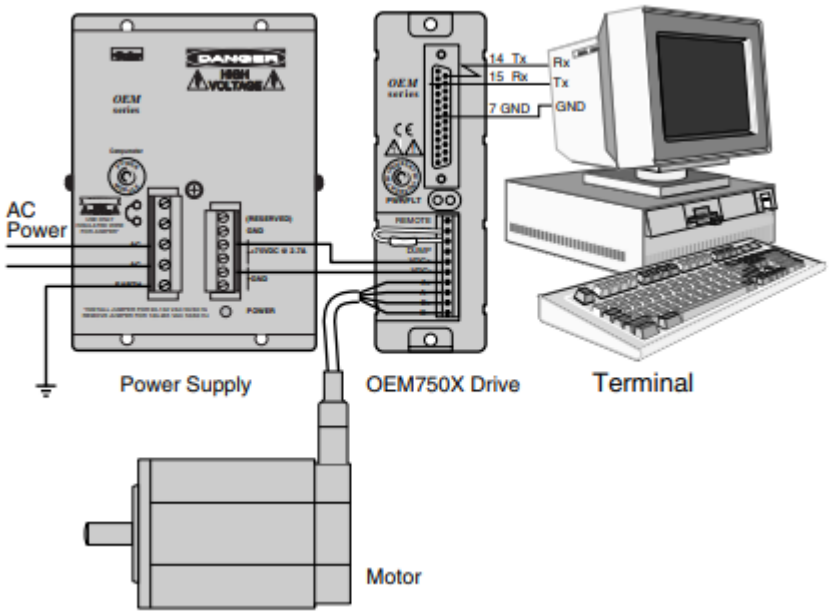
O comando "LD3" desbilita os sensores de fim de curso, sem essa configuração o motor não é acionado até a correta instalação dos sensores, "MN" define o modo normal de operação, "OSC1" define o sensor de homing como normalmente fechado devido ao uso de um transistor PNP como interruptor, "A5" configura a aceleração para 5 revolução por segundo quadrado (rps^2) e "V5" define a velocidade em 5 revolução por segundo (rps) e "ER1" define a resolução do encoder para 1 passo de encoder por revolução do motor, como a montagem do motor inclui caixas de redução essa resolução não confere com a prática, portanto o valor de 1 foi escolhido por não alterar a velocidade de rotação definida pelo comando "V".

A leitura do terminal é feita pela função *get_ reply* e elimina os caracteres referentes à decodificação, fim e retorno de linha.

O envio de cada comando deve seguir um determinado padrão, no caso, deve apresentar um fim e retorno de linha e a mensagem deve ser codificada, enviando assim uma sequência de bytes. Logo, a função *sendData* se encarrega de fazer essas alterações nos comandos antes de enviá-los.

```
def sendData(self, data):
    """Envia os comandos para o controlador"""
    data += "\r\n"
    self.ser.write(data.encode())
```

Figura 3: Representação da configuração do sistema para comunicação RS-232C com o OEM 750X.



4 Circuito

O circuito é composto pela fonte de alimentação do drive/indexador que deve fornecer uma tensão entre 24 a 75 V, o drive/indexador conectado ao computador por seu conector DB25 e ao motor pelo esquema de fios apresentado na tabela 1. O circuito descrito é apresentado na figura 3.

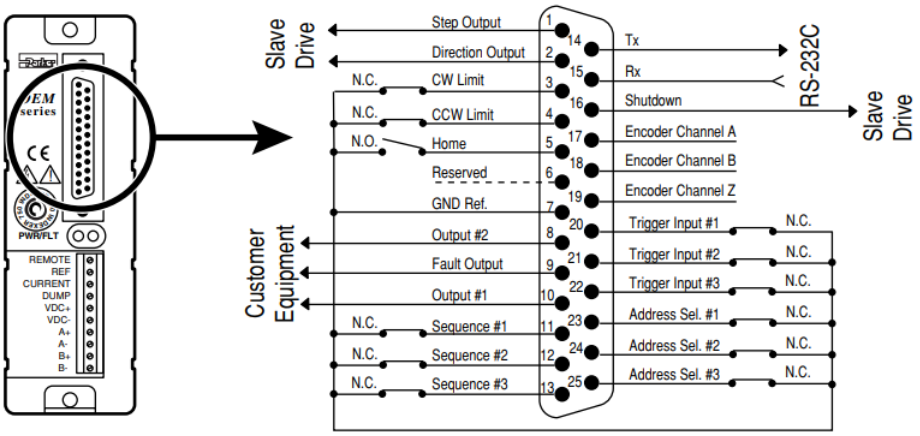
Fio	A^+	A^-	B^+	B^-
Cor	Laranja	Azul	Vermelho	Amarelo

Tabela 1: Tabela de correspondência de cores com as fases do motor 103-H8221-6241 da Sanyo Denki.

As conexões do DB25 do OEM750X permite várias funções, tais como, envio e recepção de dados por um terminal, gatilhos (*triggers*), limites, *homing*, codificadores (*encoders*). A pinagem detalhada é exibida na figura 4.

O circuito das figuras 5 e 6 representa a ligação feita para o controle do posicionamento do motor de acordo com os passos do codificador feito a partir de dois sensores fotoelétricos BGS-S08CP da Optex e um disco de

Figura 4: Representação da pinagem do conector DB25 identificando todas as funcionalidades.



MDF de raio de 100 mm e dentes de 6 mm 7. Para o sensor de *homing* foi utilizado o sensor indutivo SEIH-M12B-PS-S-L da Festo que foi acoplado à mesa de modo a permanecer em uma posição fixa e a uma distância curta o suficiente da parte da mesa que rotaciona, para a ativação do sensor. A ligação dos fios dos sensores é especificada na tabela 2. Os transistores foram utilizados como interruptores entre as entradas dos canais do codificador e o pino de referência do OEM750X.

Sensor	V+	V-	S
Fotoelétrico	Vermelhor	Marrom	Laranja
Indutivo	Marrom	Azul	Preto

Tabela 2: Fios dos sensores fotoelétricos e indutivo aplicados no circuito.

As conexões são feitas de maneira especificada, utilizando dos pinos referentes à conexão RS-232C e aos sensores de *homing*, limites e codificadores. A tabela 3 relaciona os pinos dos conectores do drive e do computador, lembrando que o TX do DB25 se conecta ao RX do DB9 e o RX do DB25 ao TX do DB9. Os pinos para os sensores de *homing*, fim de curso e codificadores são conectados diretamente aos sensores. A configuração final ficou de acordo com a figura 8, para os cabos de rede foi definida a relação entre os fios, cores e pinos descrita na tabela

Figura 5: Esquema do circuito apresentado para o controle dos sensores.

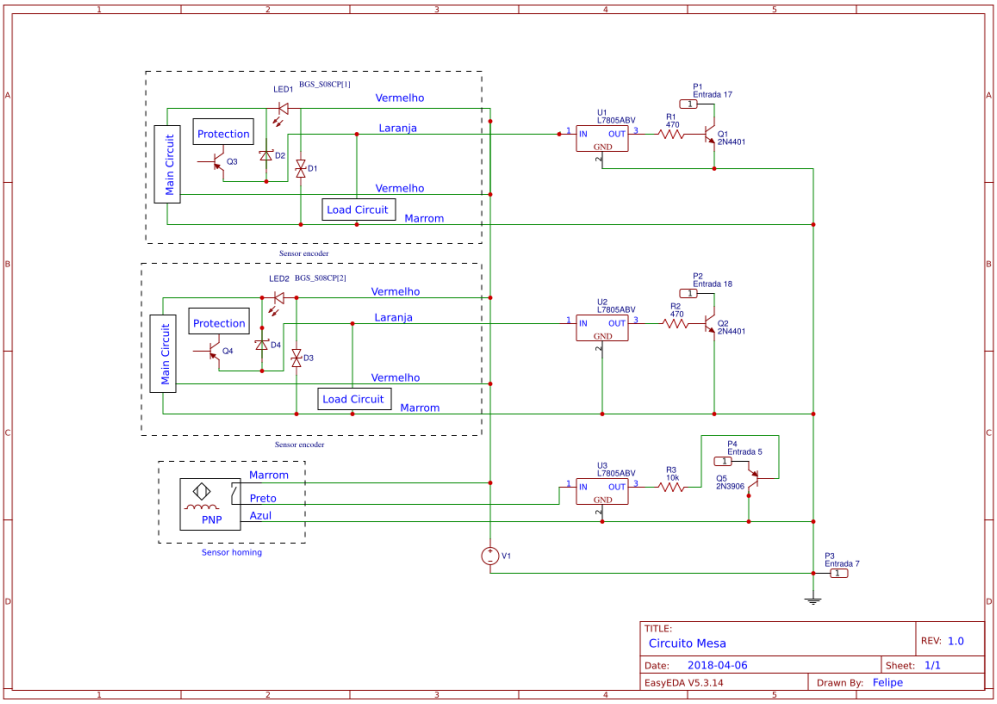


Figura 6: Esquema multifilar para o controle dos sensores.

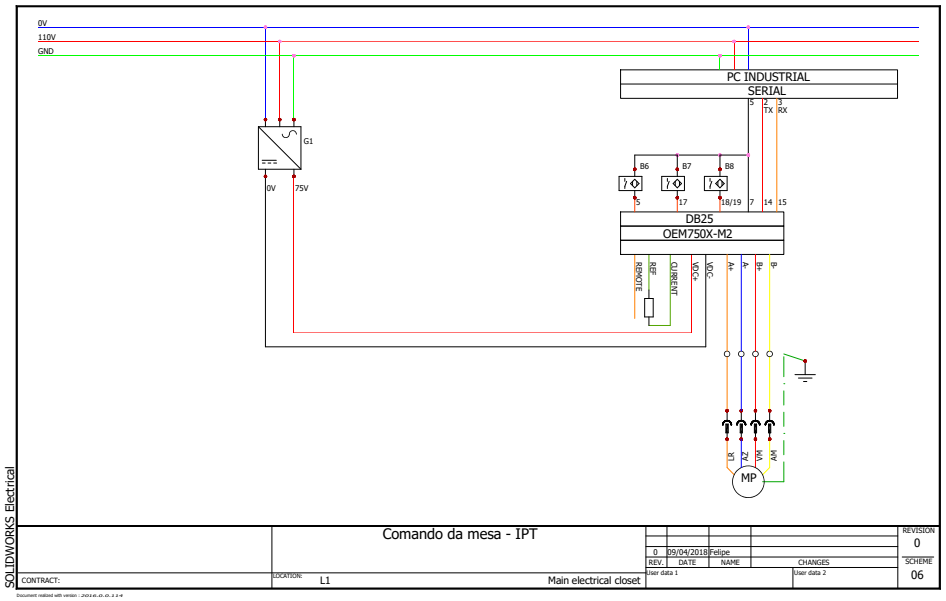


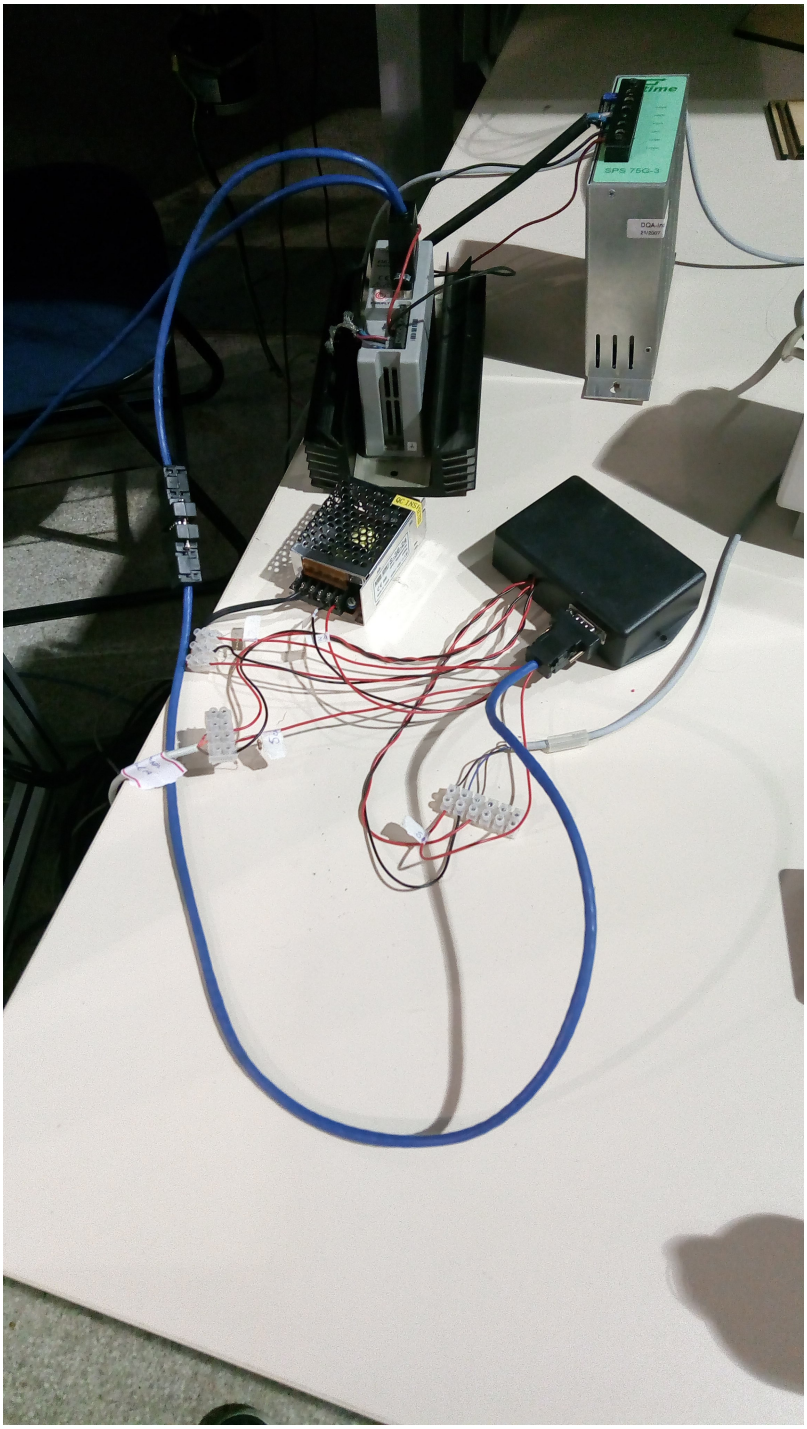
Figura 7: Sistema utilizado para o encoder.



	GND	TX	RX
DB25	7	14	15
DB9	5	2	3

Tabela 3: Relação entre as conexões no DB25 do OEM 750X e no DB9 do computador industrial.

Figura 8: Configuração final dos componentes: circuito para os sensores, fontes para sensores e indexador.



	Ch_A	Ch_B	Ch_Z	Home	S_{Ch_A}	S_{Ch_B}	S_{Home}	GND	24V
DB25	17	18	19	5	-	-	-	7	-
DB9	1	3	3	6	2	4	7	8	5
Borne	2	4	4	7	3	5	6	1	8

Tabela 4: Relação entre as conexões no DB25 do OEM 750X e no DB9 do computador industrial, Ch significa canal, enquanto o S está para o sinal do sensor.

5 Manuais

Os manuais do OEM 750X se encontram nos seguintes links:

Manual de usuário;

Guia de programação.

A referência utilizada para o motor da Sanyo Denki pode ser encontrada no link:

Datasheet.

As informações referentes aos sensores fotoelétricos e indutivo podem ser encontradas no seguinte link:

Manual do BGS-S08CP;

Informações e manual do SIEH-M12B-PS-S-L